Agentic AI Coworkers

DailyTechUpgradeAgent and **nextLevelPersonalCoachForCareerLevel**

Bernie Miao · CMPE 272, San José State University · October 16, 2026
Source code: https://github.com/EducatedBernie/agentic-ai-coworkers

---

1. Agent Goals

Agent 1: DailyTechUpgradeAgent

Purpose: help me become a better technical learner every day by doing the searching and filtering for me, so my reading time goes to high-value material.

Expected outcomes

- A daily brief of the **top 3** recent technical articles from authoritative sources: OpenAI, Anthropic, Microsoft, Google, AWS, IBM, LangChain/LangGraph, IEEE, ACM, Hugging Face, and arXiv.
- For each article: title, source, publication date, URL, a technical summary, three key takeaways, and an explanation of how it advances *my* knowledge and career goals.
- Total reading time of 15-20 minutes.
- Personalization that improves over time, using my profile, my ratings of past articles, and weak topics found by Agent 2. It never repeats an article.

Agent 2: nextLevelPersonalCoachForCareerLevel

Purpose: prepare me for my next career milestone, a Summer 2027 AI/ML or AI-product engineering internship, while keeping an entrepreneurial path open.

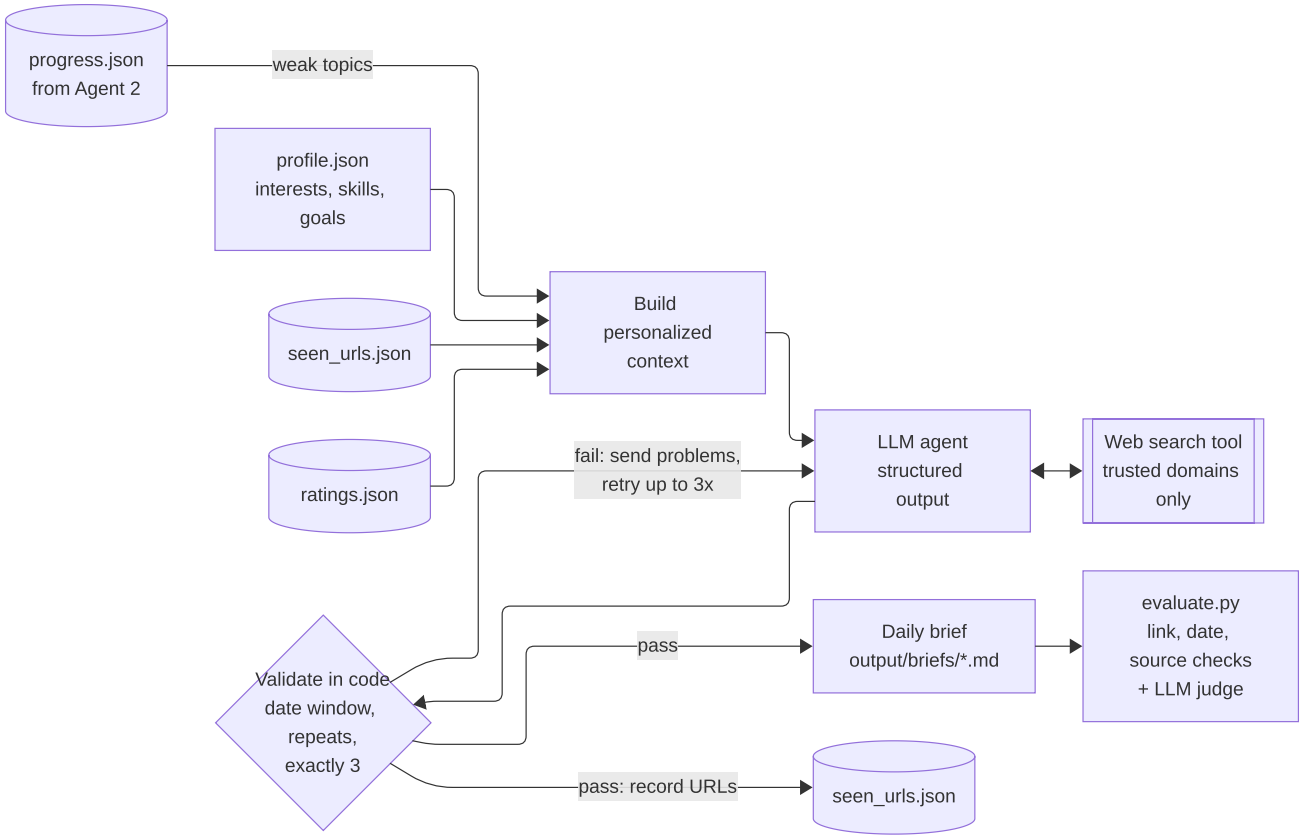
Expected outcomes

- Relevant internships, jobs, research positions, and startup programs found by live web search, each with a skills gap analysis against my profile.
- Interactive interview practice: conceptual, coding, system design, and behavioral questions grounded in real job descriptions. The coach asks one question at a time, scores each answer 1-5, gives specific feedback, and asks follow-ups that push deeper.
- Strengths, knowledge gaps, recommended resources, and a week-by-week preparation plan.
- Progress tracked across sessions, plus a **Career Readiness Report** on demand.

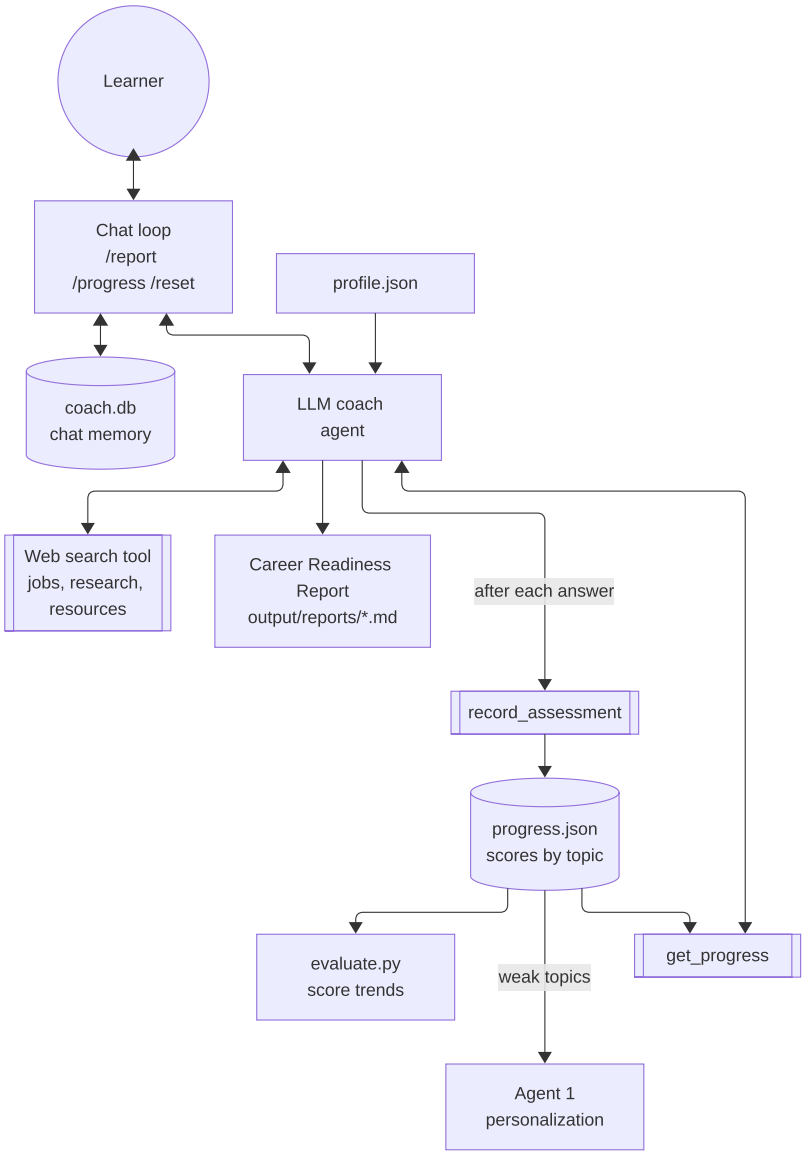
---

2. Architecture

Agent 1 architecture



Agent 2 architecture



Component	Implementation
Framework	OpenAI Agents SDK (Python) on the OpenAI Responses API
Agent 1 tools	Hosted web search, restricted to a trusted-domain allowlist
Agent 2 tools	Hosted web search; `record_assessment()` and `get_progress()` Python function tools
Data sources	Live web (engineering blogs, research sites, job boards), `profile.json`, my answers and ratings
Short-term memory	SQLite-backed conversation session (`data/coach.db`) that persists across coaching runs

Component	Implementation
Long-term memory	`progress.json` (scores by topic), `ratings.json` (article feedback), `seen_urls.json` (no repeats)
Shared memory	Topics Agent 2 scores below 3/5 are passed to Agent 1, which then picks articles on them

Agent 1 workflow

1. **Build context.** Load my profile, the last 100 recommended URLs, my recent ratings, and my weak topics from the coach's progress file.
2. **Search and select.** The agent searches trusted domains and returns a typed `DailyBrief` object (structured output) with exactly 3 articles.
3. **Validate in code.** Each date is parsed and checked against the lookback window, and every URL is checked against history. If anything fails, the specific problems are sent back to the agent, which supplies replacements (up to 3 attempts).
4. **Publish.** Save the Markdown brief and the raw JSON, and add the URLs to history.

Agent 2 workflow

1. On startup the coach calls `get_progress()` and greets me with where I stand.
2. **Opportunities:** web search finds postings, and the coach compares their requirements with my skills.
3. **Drills:** the coach asks me a question, waits for my answer, evaluates it, calls `record_assessment()`, then asks a harder follow-up.
4. `/report`**:** the coach reads all progress data and writes the Career Readiness Report.

---

3. Working Prototype

I built the prototype in about 600 lines of Python across four files:

File	Role
`daily_tech.py`	Agent 1. Domain-filtered search, structured output, validation-and-retry loop
`coach.py`	Agent 2. Interactive command-line chat with commands `/report`, `/progress`, `/reset`, `/quit`
`evaluate.py`	Automated accuracy checks, LLM-judge scoring, user ratings, progress trends
`common.py`	Shared paths, atomic JSON persistence, configuration

```
pip install -r requirements.txt
cp .env.example .env               # add OPENAI_API_KEY
python daily_tech.py               # Agent 1: today's brief
```

```
python coach.py                    # Agent 2: interactive coaching
python evaluate.py judge           # LLM-judge scores for new briefs
python evaluate.py                 # evaluation report
```

To run Agent 1 every day, I schedule it with cron, e.g. `0 8 * * * .venv/bin/python daily_tech.py`.

My design decisions

- **I used structured output** (Pydantic models), which guarantees every brief has the required fields and gives the evaluator data it can parse.
- **I restricted search to trusted domains**, which makes source authority a hard limit the model cannot override.
- **I validate in code** rather than trusting the prompt. My first live run showed why: the model ignored the date window and returned a March 2026 article. The validation loop caught it and the agent replaced it.
- **I had the coach write scores through a tool call**, so my progress is structured data, not something buried in chat text.

---

4. Sample Results

Full outputs are in Appendices A-C and in the repository's `output/` folder.

Daily technical recommendations (Oct 16, 2026)

Theme: *reliable agentic applications that combine efficient tool selection, trace-aware evaluation, and persistent memory.*

#	Article	Source	Date
1	TRACER: Trajectory-Aware Tool Retrieval for LLM Agents	arXiv	2026-10-15
2	Four evaluation engineering lessons learned from building agent plugins	Google Cloud Blog	2026-10-16
3	How to implement long-term AI agent memory in AlloyDB and Memorystore for Valkey	Google Cloud Blog	2026-10-02

My interactive career coaching session

- **Opportunities found:** Google Forward Deployed Engineering Intern (Sunnyvale), DoorDash ML Intern, Master's (SF/Sunnyvale), and Veeam AI/ML Intern (San Jose), each with a skills gap analysis.

- **Drills I answered:** restaurant recommender system design (3/5), RAG support assistant design (3/5), combined recommender + RAG design (4/5), and a Python top-k coding problem (4/5).
- After each of my answers the coach gave concrete feedback and a harder follow-up. For example, after my RAG answer it pointed out that I missed hybrid search, reranking, citation verification, and prompt-injection defenses. I used those ideas in my next answer and scored higher.
- **My readiness report:** overall readiness 68/100, a gap list, linked resources, a 4-week plan, and a measurable milestone (ship a recommender with grounded RAG explanations by Nov 13, 2026).

---

5. Evaluation

How I measure effectiveness, accuracy, and progress

Dimension	Metric	Method
Accuracy (Agent 1)	Link validity	HTTP request to every URL, to catch made-up links
	Freshness	Published date is inside the 14-day window
	Source authority	URL host is on the trusted-domain allowlist
	Novelty	URL was never recommended before
	Format compliance	Exactly 3 articles per brief
Effectiveness (Agent 1)	Relevance, depth, actionability (1-5 each)	An independent LLM judge opens each article and scores it against my profile
	Usefulness (1-5)	My own rating per article (`evaluate.py rate`), which also feeds personalization
My progress (Agent 2)	Score per topic over time	First vs. latest score per topic, and early vs. late average across all answers
	Gap closure	Topics below 3/5 should rise after targeted reading and practice
Coaching quality (Agent 2)	Fair grading, real postings	I spot-check scores and check that cited job links load

Results (Oct 16, 2026)

Metric	Result
Link loads	100% (3/3)
Date within window (after validation)	100% (3/3)
From trusted domain	100% (3/3)

Metric	Result
No repeats	100% (3/3)
Exactly 3 articles	100% (1/1)
LLM-judge relevance / depth / actionability	5.0 / 4.0 / 4.0 out of 5
My coach scores, first half vs. second half of answers	3.0 → 4.0
Cited job and resource links	Google and Built In postings and all checked resource links load; Indeed (DoorDash posting) blocks automated checks, so unverified

What I found

Running these checks on my first live outputs, I found two real defects and fixed both:

1. **Dates out of window.** Agent 1 returned a September and then a March article despite being told to use only the last 14 days, and it formatted dates inconsistently. I fixed it with code-side date parsing and the validate-and-replace loop. Freshness went from 2/3 to 3/3.
2. **Placeholder score.** Agent 2 recorded a 1/5 score when it *asked* a question, before any answer existed, which would have distorted the progress trend. I fixed it by tightening the tool and agent instructions, and I removed the bogus entry.

Limitations

- I have one day of briefs so far and haven't added my own usefulness ratings yet, so the LLM judge measures effectiveness for now. Trends in ratings and novelty need several days of runs.
- An LLM grading my answers can be inconsistent. I told the coach not to inflate scores and I spot-check outliers, but scores are a signal, not ground truth.

Appendix A: Daily Tech Brief (Oct 16, 2026)

Daily Tech Brief: 2026-10-16

Theme: Reliable agentic applications combine efficient tool selection, trace-aware evaluation, and persistent memory.
Total reading time: ~20 min

Suggested order: Read TRACER first for efficient tool use, then the evaluation article for reliability, and finish with the memory architecture piece for cloud system design. Total: 20 minutes.

1. TRACER: Trajectory-Aware Tool Retrieval for LLM Agents

arXiv | 2026-10-15 | ~7 min
https://arxiv.org/abs/2610.21267

TRACER retrieves a small, contextually relevant subset of tools instead of exposing an agent to an entire registry. It indexes prior tool-call sequences in a trie and combines empirical usage patterns with conversation-aware retrieval. Across six environments, it improved tool-selection accuracy in 21 of 24 model-environment pairs while substantially reducing prompt-token usage.

Key takeaways

- Large tool registries can hurt both cost and tool selection.
- Prior execution trajectories can guide dynamic tool retrieval.
- Always design fallbacks for unseen tool-call sequences and cold-start deployments.

Why it matters for you: This is directly relevant to your beginner-level agent-building goals and to AI tools for students or small businesses, where controlling tool count, latency, and API cost matters.

2. Four evaluation engineering lessons learned from building agent plugins

Google Cloud Blog | 2026-10-16 | ~6 min
https://cloud.google.com/blog/topics/developers-practitioners/four-eval-engineering-lessons

This article argues that agent evaluation must inspect execution traces rather than only final answers. Using a Google Cloud developer plugin as a case study, it explains how tool-call logs, repeatable test suites, and workflow-specific rubrics reveal whether an agent actually performed required actions. The lessons provide a practical foundation for evaluating tool use and agent reliability.

Key takeaways

- Grade tool-call trajectories, not only summaries.
- Use repeatable, workflow-specific evaluation suites.
- Treat evaluation engineering as a core development discipline.

Why it matters for you: For a Summer 2027 AI/ML or AI-product internship, this offers a practical portfolio pattern: build an agent, log its actions, define rubrics, and demonstrate reliability improvements.

3. How to implement long-term AI agent memory in AlloyDB and Memorystore for Valkey

Google Cloud Blog | 2026-10-02 | ~7 min
https://cloud.google.com/blog/products/databases/implementing-long-term-ai-agent-memory-in-alloydb-and-memorystore/

This post presents a two-tier memory architecture for long-running agents: fast short-term state in Memorystore for Valkey and durable semantic memory in AlloyDB AI. It explains how agents can retain user preferences and workflow context across sessions while controlling latency, token usage, and access boundaries. The design connects agent memory to practical database and cloud architecture choices.

Key takeaways

- Separate short-lived conversational state from durable user memory.
- Use database-backed memory for multi-session workflows rather than endlessly expanding prompts.
- Treat persistence, retrieval, and access controls as architectural concerns.

Why it matters for you: This strengthens your beginner cloud and system-design skills and maps well to useful products for students or small businesses, such as personalized study, scheduling, or customer-support agents.

Rate this brief: `python evaluate.py rate 2026-10-16 <s1> <s2> <s3>` *(1-5 each)*

Full file: output/briefs/2026-10-16.md

Appendix B: Coaching Session Transcript

Coaching session: 2026-10-16

Coach: Hi Bernie—your practice history is empty, so we'll establish a baseline. Would you like to **find Summer 2027 opportunities**, **start an interview drill**, or **review a preparation plan**?

You: Find me 3 Summer 2027 AI/ML or AI product internships in the Bay Area or remote that are open or opening soon, and tell me what skills I'm missing for each.

Coach: I found three strong matches. **Apply quickly: postings can close or fill on a rolling basis.**

1. Google — Forward Deployed Engineering Intern, BS/MS

- **Location:** Sunnyvale, CA
- **Deadline: November 20, 2026**; rolling review
- **Link:** Google Careers posting (google.com)
- **Fit:** Excellent match for AI-product engineering, agents, RAG, and cloud.

You have: Python, ML fundamentals, beginner LLM/agent interest, MS program.
Missing or needs proof:

- Production AI system shipped end-to-end
- GCP/cloud architecture
- Vector databases and production RAG
- Multi-agent frameworks such as LangGraph, CrewAI, or ADK
- Evaluation, observability, tracing, latency, and cost optimization
- Customer-facing technical discovery

2. DoorDash — Machine Learning Intern, Master's, Summer 2027

- **Location:** San Francisco, Sunnyvale, New York, or Seattle
- **Status:** Applications accepted on an ongoing basis
- **Link:** DoorDash posting (indeed.com)
- **Fit:** Strong match for your recommender-systems and applied-ML interests.

You have: MS degree path, Python, SQL, ML fundamentals, recommender-systems interest.
Missing or needs proof:

- Strong PyTorch/TensorFlow/MLflow experience
- Practical ranking or recommendation project
- ML research experience and experimental rigor
- Big-data/analytics experience

- Stronger systems-language or production-software depth
- Publications are preferred, though not mandatory

3. Veeam — AI/ML Intern, Summer 2027

- **Location:** San Jose, CA; in-office five days/week
- **Program:** June–August 2027
- **Link:** Built In San Francisco posting (builtinsf.com)
- **Fit:** Good match if you want model training, LLM evaluation, and ML research tooling.

You have: ML fundamentals, Python, graduate study, LLM/AI interest.
Missing or needs proof:

- Hands-on deep-learning training in PyTorch or TensorFlow
- Diagnostics for overfitting, gradients, convergence, leakage, and generalization
- Probability, statistics, optimization, and experimentation depth
- Clean modular/OOP software engineering
- LLM fine-tuning/evaluation
- MLflow/W&B, LoRA/QLoRA, mixed precision, or distributed training
- A reproducible research project or paper reproduction

Priority recommendation: Apply to **Google and DoorDash first**, then build one portfolio project combining **RAG + vector database + agent workflow + evaluation/tracing**. That single project would directly address many of your largest gaps.

You: Start a drill based on the DoorDash ML intern and Google FDE postings. Mix recommender systems, RAG, and system design.

Coach: ### Drill 1 — Recommender Systems + System Design

Design a DoorDash home-screen recommendation system that ranks restaurants for a user.

Cover:

1. Objective and success metrics
2. Candidate generation
3. Ranking features and model
4. Cold-start handling
5. Offline and online evaluation
6. Production trade-offs: latency, freshness, cost, and reliability

Answer as if in an interview.

You: The goal is to get users to order, so I'd optimize click-through and conversion rate. For candidates I'd filter restaurants that are open and deliver to the user's address, then use collaborative filtering, like matrix factorization on user-restaurant orders, to get maybe a few hundred candidates. For ranking I'd

train a gradient boosted tree like XGBoost on features like past orders, cuisine preference, distance, delivery time estimate, rating, price, and time of day. For cold start I'd show popular restaurants nearby. Offline I'd measure precision@k and NDCG, then A/B test online on conversion. For latency I'd precompute the embeddings and cache results.

Coach: Score: 3/5 — solid foundation.

What you did well

- Correct two-stage approach: candidate generation then ranking.
- Relevant features: cuisine, distance, ETA, price, time of day.
- Appropriate cold-start fallback and online A/B testing.
- Mentioned caching and precomputation for latency.

What was missing

- Optimize more than CTR: completed orders, contribution margin, retention, and cancellations.
- Handle selection bias from historical orders and displayed items.
- Include freshness: restaurants may close or have changing ETAs.
- Add guardrails: delivery quality, diversity, fairness, and cancellation rate.
- Specify latency targets and failure fallbacks.

Better architecture

Use hard eligibility filtering → popularity/CF/content retrieval → feature-rich ranking → business-rule reranking. Track NDCG/recall offline, then A/B test completed orders and long-term retention with guardrails.

Follow-up Drill 2 — RAG + System Design

Google's FDE role may require building a customer-facing AI assistant. Design a RAG system for DoorDash support agents that answers questions using restaurant policies, delivery records, and company documentation.

Explain:

- Document ingestion and chunking
- Retrieval and reranking
- How you prevent unsupported answers
- Evaluation metrics
- Latency, privacy, and production failure handling

You: I'd split the documents into chunks of around 500 tokens, embed them with an embedding model and store them in a vector database like Pinecone. When an agent asks a question I embed the

question, get the top 5 most similar chunks and put them in the prompt for the LLM. To prevent made up answers I'd tell the LLM in the system prompt to only answer from the context and say I don't know otherwise. For evaluation I'd have people check if the answers are correct. For privacy I'd not include customer info. I'm not really sure about reranking.

Coach: Score: 3/5 — good baseline, but not production-ready yet.

Strong points

- Sensible chunking and vector retrieval pipeline.
- Correctly included abstention instructions.
- Recognized privacy concerns and human evaluation.

Key improvements

- Add **metadata filters**: region, policy version, restaurant, date, and user permissions.
- Use **hybrid search**: BM25 for exact terms plus embeddings for semantic matches.
- Rerank the top 20–50 results with a cross-encoder, then send the best 5–8 chunks to the LLM.
- Require citations and verify that claims are supported by retrieved text.
- Evaluate retrieval recall@k, answer faithfulness, correctness, latency, and abstention quality.
- Protect against prompt injection in documents and enforce authorization before retrieving customer data.
- Add fallback behavior: search escalation, human handoff, and stale-document detection.

Drill 3 — Integrated System Design

Design an AI-powered restaurant recommender that uses both recommendation models and RAG.

Example: a user asks, **“Find me vegetarian Thai restaurants under $30 that can deliver within 30 minutes, and explain why you recommend them.”**

Describe the end-to-end request flow, including:

- Intent and constraint extraction
- Candidate retrieval and ranking
- Structured data versus RAG usage
- Tool/API calls
- Response generation and grounding
- Evaluation and latency targets

You: First an LLM call with structured output extracts the constraints: cuisine=Thai, dietary=vegetarian, max_price=30, max_eta=30 min. Those are hard filters, so they go to the restaurant database as a SQL/API query, not to RAG, because prices and ETAs change constantly. The eligible restaurants go through the same ranker from drill 1 for personalization. Then for the top 5 I'd use RAG over menus and

reviews, with hybrid search and a reranker like you said, to find vegetarian dishes and evidence to explain the recommendation. The LLM writes the answer citing the menu items and reviews, and I'd check every restaurant it mentions is actually in the filtered list. If no restaurants match I'd relax the ETA and say so. For evaluation: constraint satisfaction rate, which should be 100%, citation faithfulness, and conversion in an A/B test. Latency target maybe 2 to 3 seconds, so I'd run the RAG lookups in parallel and cache menu embeddings.

Coach: Score: 4/5 — strong interview answer.

What you did well

- Correctly separated dynamic structured data from RAG.
- Used hard constraints before personalized ranking.
- Added hybrid retrieval, reranking, citations, and result validation.
- Included graceful constraint relaxation and a realistic latency target.
- Chose strong evaluation metrics.

To reach 5/5

Mention:

- Clarifying ambiguous “under $30” and “vegetarian” definitions.
- Authorization and prompt-injection defenses for reviews/menus.
- Timeouts and partial-result fallbacks for APIs and RAG.
- Tracing: extraction errors, retrieval recall, ranking scores, citation failures, and latency.
- A deterministic parser could handle simple constraints without an LLM.

Overall drill performance: recommender systems **3/5**, RAG **3/5**, integrated system design **4/5**. Your strongest area is architectural decomposition; focus next on production reliability, evaluation depth, and security.

You: Give me one more: a quick Python coding question.

Coach: ### Quick Python Coding Drill

Implement:

```
def top_k_recommendations(scores: dict[str, float], k: int) -> list[str]:
    ...
```

Requirements:

- Return the restaurant IDs with the highest scores.
- Return at most `k` IDs.
- Break score ties alphabetically by restaurant ID.

- Do not mutate the input dictionary.
- Raise `ValueError` if `k < 0`.

Example:

```
scores = {"r3": 0.9, "r1": 0.9, "r2": 0.8}

top_k_recommendations(scores, 2)
### ["r1", "r3"]
```

Explain the time and space complexity.

You: ```python import heapq

def top_k_recommendations(scores: dict[str, float], k: int) -> list[str]: if k < 0: raise ValueError("k must be non-negative") return [rid for rid, _ in heapq.nsmallest(k, scores.items(), key=lambda kv: (-kv[1], kv[0]))] ``` nsmallest with key (-score, id) gives highest score first and breaks ties alphabetically. It does not mutate the dict. Time is O(n log k) because it keeps a heap of size k, space is O(k). If k >= n it just sorts, so O(n log n).

Coach: Score: 4/5 — interview-ready solution.

Your implementation is correct and handles validation, tie-breaking, and non-mutation cleanly. Complexity analysis was accurate, including the `k >= n` case.

Minor note: explicitly mention that `k = 0` returns `[]`.

You: /report

Coach: (generated report: readiness-2026-10-16.md)

Full file: output/sessions/2026-10-16_sample.md

Appendix C: Career Readiness Report

Career Readiness Report — Bernie

1. Target Roles and Overall Readiness

Target roles: AI/ML Engineer Intern and Software Engineer Intern — AI Products, targeting Summer 2027 in the Bay Area or remotely.

Overall readiness: 68/100. You demonstrate solid Python fundamentals and promising system-design instincts, especially in separating structured data from RAG and designing multi-stage recommendation pipelines. Your main limitations are limited demonstrated experience with production ML/RAG systems, evaluation methodology, security, cloud deployment, and recommender-system trade-offs. You are directionally strong but need one polished end-to-end project and deeper interview practice before being fully competitive.

2. Opportunities Found

Organization	Role	Location	Link	Main missing skills
Google	Forward Deployed Engineering Intern, BS/MS — Summer 2027	Sunnyvale, CA	Google Careers	Production AI systems, GCP, vector databases, agent frameworks, evaluation/observability, customer-facing technical discovery
DoorDash	Machine Learning Intern, Master's — Summer 2027	San Francisco/Sunnyvale and other locations	DoorDash posting	PyTorch/TensorFlow, MLflow, recommender/ranking project, experimentation, big-data systems, stronger production engineering
Veeam	AI/ML Intern — Summer 2027	San Jose, CA	Built In posting	Deep learning, PyTorch/TensorFlow, ML diagnostics, statistics/optimization, LLM evaluation/fine-tuning, reproducible research

3. Technical Assessment

Topic	Attempts	Average	Trend
Python	1	4.0/5	4 → 4
RAG	1	3.0/5	3 → 3
Recommender systems	1	3.0/5	3 → 3

Topic	Attempts	Average	Trend
System design	1	4.0/5	4 → 4

4. Strengths

- Clear Python implementation and complexity reasoning.
- Good architectural decomposition.
- Correct separation of structured databases from unstructured RAG.
- Strong understanding of two-stage recommendation systems.
- Practical awareness of caching, latency, parallelism, and fallbacks.
- Comfortable discussing A/B testing and business metrics.
- Good instinct for evidence-based AI responses.

5. Knowledge Gaps

- RAG reranking, retrieval recall, citation verification, and automated evaluation.
- Prompt injection, authorization, privacy, and secure tool use.
- Recommendation bias, exposure bias, diversity, fairness, freshness, and guardrails.
- Deep learning frameworks and training workflows.
- ML experiment tracking and reproducibility.
- Cloud deployment and observability.
- Production reliability: timeouts, retries, partial failures, and SLOs.
- More practice with ambiguous requirements and behavioral communication.

6. Recommended Resources

- Google Recommendation Systems course — candidate generation, scoring, reranking, and embeddings. (developers.google.com)
- Google recommendation retrieval — embeddings and approximate nearest-neighbor retrieval. (developers.google.com)
- Google recommendation reranking — freshness, diversity, and fairness. (developers.google.com)
- Google Cloud RAG reference architecture — ingestion, metadata, serving, freshness, and deployment. (docs.cloud.google.com)
- Google Cloud RAG architecture guides — production alternatives and CI/CD patterns. (docs.cloud.google.com)
- Google Cloud Well-Architected Framework — security, reliability, performance, and cost. (docs.cloud.google.com)
- Project: build a **personalized restaurant recommender with RAG explanations**, using Python, PostgreSQL, a vector database, FastAPI, Docker, and a cloud deployment.
- Project evaluation: create a labeled test set measuring constraint satisfaction, Recall@K, NDCG@K, citation faithfulness, latency, and cost.

7. Four-Week Preparation Plan

Week 1 — Recommender Systems

- Complete Google's recommendation-systems lessons.
- Implement popularity, content-based, and matrix-factorization baselines.
- Add candidate generation, ranking, and reranking.
- Practice one recommender-system interview question.

Week 2 — RAG Engineering

- Build ingestion, chunking, metadata filtering, hybrid retrieval, and reranking.
- Add citations and abstention behavior.
- Create a 50-question evaluation set.
- Measure retrieval recall, answer correctness, faithfulness, latency, and cost.

Week 3 — Production System Design

- Deploy the project behind a FastAPI service.
- Add Docker, logging, tracing, retries, timeouts, and caching.
- Document privacy, authorization, prompt-injection defenses, and failure modes.
- Practice two timed system-design interviews.

Week 4 — Interview and Application Readiness

- Complete three Python coding drills.
- Complete two recommender/RAG design drills.
- Prepare STAR stories for project ownership, debugging, ambiguity, and teamwork.
- Polish GitHub README, architecture diagram, metrics, demo, and resume bullets.
- Begin targeted applications and outreach.

8. Next Milestone and Measurement

Milestone: Ship and explain a production-style restaurant recommender with grounded RAG explanations by **November 13, 2026**.

Measure success by:

- At least **5/5** on an integrated recommender/RAG system-design drill.
- At least **4/5** on two additional Python problems.
- Retrieval Recall@5 of **≥80%** on the evaluation set.
- Constraint satisfaction of **≥98%**.
- Citation faithfulness of **≥90%**.
- A deployed demo with documented latency, cost, security, and failure handling.

Full file: output/reports/readiness-2026-10-16.md